



Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

As lifetime increases:
Every additional time step appends a new percept to the historical log, causing the total number of possible percept sequences to grow exponentially. This leads to a combinatorial explosion, making table lookups and searches completely intractable over time.

2. (Understand) Look at the code you wrote for your SimpleReflexAgent . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The Condition-Action Rules in the SimpleReflexAgent are the if statements inside the sense_and_act() method.
if percept["food_here"]: return "Stay" – If food is present, the agent stays to collect it.
if percept["wall_ahead"]: – If a wall is ahead, the agent turns to avoid it based on its current direction.
return percept["facing"] – If there is no food or wall, the agent continues moving in the same direction.

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The SimpleReflexAgent got stuck in an infinite loop because it had partial observability and no percept history. It could only see the current percept (such as food or a wall) and could not remember previous states or actions. As a result, when it encountered the same situation repeatedly, it kept choosing the same action, causing it to loop indefinitely instead of finding a new path.

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The ModelBasedAgent uses a simple transition model by storing its previous actions in action_history and last_action, allowing it to avoid repeating the same turns and reducing loops. Its sensor model updates this internal state using the current percept (such as food_here and wall_ahead) together with the remembered actions to choose a better next action.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.